

SETTINGS

Claude Code settings

Copy page

Change Claude Code settings, pick the scope a key belongs in, verify the change, and learn which value Claude Code uses when a key is set in several places.

! This page covers Claude Code running on your machine: the terminal, the [VS Code](#) and [JetBrains](#) extensions, and the [desktop app](#), which all read the same settings files. A cloud session on [Claude Code on the web](#) runs on a different machine and reads only some of them; see [Settings in cloud sessions](#).

Settings are the JSON keys that change how Claude Code behaves: which model it starts with, what it can run without asking, which files it can't read, how it looks in your terminal, and what your organization enforces.

Claude Code reads settings from JSON settings files such as `~/.claude/settings.json`. It looks for them in a few locations, and **the file it reads a setting from decides who the setting applies to**.

Use this page to pick the settings file that reaches the people you want a setting to apply to, change a setting and confirm it applied, and see which value Claude Code uses when the same key is set in more than one file.

The **settings reference** lists every key you can set, with the file you set it in, its type, and its default. **Configure permissions** covers what Claude Code can run without asking and how to write `allow`, `ask`, and `deny` rules.

Settings files and who they affect

Claude Code reads settings from four files, and an organization can also deliver managed settings from the claude.ai console. Each source has a scope: the set of people and projects a setting saved in it applies to, whether that's just you, everyone in a project, or everyone in your organization.

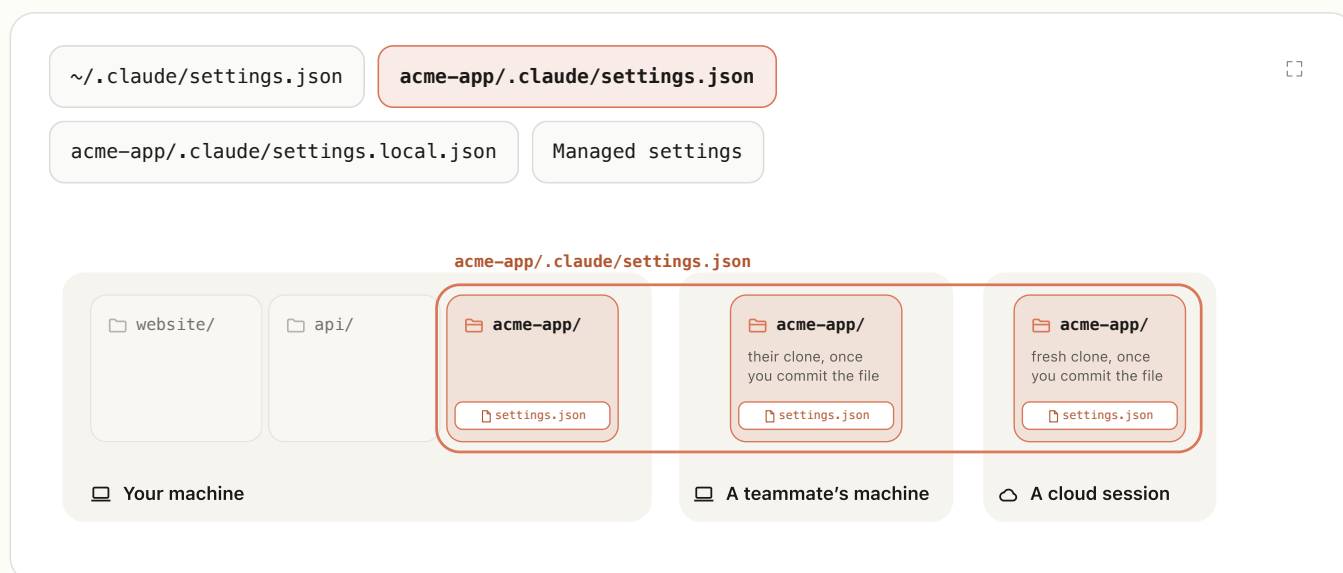
Scope	File	Who it affects	Use it for
User	<code>~/.claude/settings.json</code>	You, in every project on this machine	Personal preferences: theme, editor mode, default model, your own permission rules
Shared project	<code>.claude/settings.json</code>	Everyone working in the folder that contains it. In a git repository, commit it so teammates get it	Team permissions, hooks, plugins, and the environment variables the project needs
Project local	<code>.claude/settings.local.json</code>	You, in this one project only. Claude Code keeps it out of git when it creates the file; if you create it by hand, add it to <code>.gitignore</code> yourself	Personal overrides for one project, and testing before you share

In the File column, `~/ .claude` is the `.claude` folder in your home directory, and a bare `.claude` is the `.claude` folder inside your project.

Compare the scope of each settings file

Suppose you have three projects on your machine, `website/`, `api/`, and `acme-app/`, a teammate has their own clone of `acme-app/`, and you start a cloud session on `acme-app/`.

The graphic below shows which of those folders a setting applies in when you start Claude Code from them. Click a settings file to see the folders it reaches.



- `~/ .claude/settings.json` : every project on your machine, and nothing on your teammate's or in the cloud session
- `acme-app/.claude/settings.json` : your `acme-app/` . It reaches your teammate's clone and the cloud session only if you commit the file to version control; until you do, it's a file on your disk like any other and nobody else has it
- `acme-app/.claude/settings.local.json` : your `acme-app/` only. Claude Code adds it to your global git excludes the first time it writes the file, so it stays out of your commits; if you create the file by hand, add it to .gitignore yourself
- **Managed settings**, whether a `managed-settings.json` file, an MDM policy, or server-managed settings from the claude.ai console: every project on every machine your organization deploys it to, or that you sign in to with your organization account. Only server-managed settings reach the cloud session

- **Managed:** your organization deploys it. You don't create or edit it.
- **Shared project:** a project that already uses Claude Code may have one committed. If not, create it at `.claude/settings.json` in the project folder.
- **User and Project local:** create them yourself, or let Claude Code create them. It writes `~/.claude/settings.json` the first time you change an option in the `/config` menu that it stores in user settings, such as the theme, and `.claude/settings.local.json` the first time you give a standing approval on a permission prompt, such as "Yes, and don't ask again" for a Bash command. A few `/config` options, including **Show tips**, save to `.claude/settings.local.json` instead of the user file.

① On Windows, `~/.claude` means `%USERPROFILE%\ .claude` . To keep the home-directory files somewhere else, set `CLAUDE_CONFIG_DIR` ; Claude Code then stores your settings, session history, and plugins there instead.

Claude Code also keeps a fifth file, `~/.claude.json` , that it writes for itself; you don't need to edit it. It holds your sign-in session, **MCP server** configurations, per-project state such as trust decisions, and the **global config keys** that `/config` writes for you.

Share settings with your team

Commit `.claude/settings.json` so everyone who clones the repository gets the same permissions, hooks, telemetry, and plugins. Each teammate can still override it for themselves in their own `.claude/settings.local.json` , so personal exceptions don't need a commit. For a complete team file, see [a team's shared settings](#).

Some of what you commit waits until each teammate **trusts the folder**, and a few keys never take effect from a repository file; [Troubleshoot a setting that doesn't apply](#) covers both.

Keep personal settings out of a repository

To change a setting for yourself in one project without changing it for your teammates, save it in `.claude/settings.local.json` inside the project. Claude Code applies that file over the committed `.claude/settings.json` , so if your team's file sets `"model": "claude-sonnet-5"` and you want Opus, put `"model": "claude-opus-4-8"` in your local file and only your sessions change.

Three things to know about the local file:

- **Claude Code writes it too.** When Claude asks permission to run a Bash command and you choose "Yes, and don't ask again", Claude Code saves that **permission approval** here as an `allow` rule.

`$XDG_CONFIG_HOME/git/ignore` , or `~/.config/git/ignore` when `XDG_CONFIG_HOME` is unset. If you created the file by hand and Claude Code hasn't written to it yet, add it to `.gitignore` yourself.

- **Its allow rules don't wait for trust while the file stays untracked.** Because the file is yours and not the repository's, Claude Code applies its `allow` rules without the **workspace trust** step it requires for the committed file. If the file is tracked by git, the trust step applies to it too; see [When your local settings file needs trust](#).

Where Claude Code keeps the local file in a git repository

When Claude asks permission to run a Bash command and you choose “Yes, and don't ask again”, Claude Code saves that approval as an `allow` rule in `.claude/settings.local.json` . If you start Claude Code in a subdirectory of a git repository, it reads and writes that file at the repository root and applies the approval across the whole repository. In a **worktree**, it uses the file at the main checkout's root.

Two rules qualify the root location:

- **When the file stays with `.claude/settings.json` instead:** outside a git repository, when the repository root is your home directory, on Windows, or when the repository root or its `.git` or `.claude` entry isn't owned by your user.
- **Paths in the file don't anchor at the repository root:** a permission rule that starts with `/` or a relative sandbox path **anchors at the session's primary working directory** instead.

Before v2.1.211, Claude Code kept the file in the starting directory. It still reads a file an earlier version left there alongside the root file; where both set the same key, the root's value applies, and permission rules from both files apply. The Agent SDK's `resolveSettings()` helper always reads the file from the starting directory.

Claude Code reads the shared `.claude/settings.json` from the session's **primary working directory**, so to use a file committed at the repository root, start Claude Code there. After you **move the session with** `/cd` , Claude Code reads both project files from the new directory instead, placing the local file by the same rules. Reading them from the directory you moved to requires Claude Code v2.1.246 or later.

Check what your organization enforces

If your organization manages Claude Code, some settings are decided for you and nothing you put in your own files changes them. To see which, run `/status` : the `Setting sources` line names the managed source that applies to you. Managed settings apply wherever Claude Code runs on this machine; [What a developer can change](#) covers local admin rights and tools other than Claude Code.

Managed settings reach you through the **delivery mechanisms** on the managed settings page, most commonly:

- **Server-managed settings**, which Claude Code fetches from the claude.ai admin console or a self-hosted **Claude apps gateway**
- MDM or OS-level policies, and `managed-settings.json` files in a system directory

managed settings from the Claude AI admin console, and it reads policy deployed to your device unless your organization's Claude Desktop configuration sets `requireCoworkFullVmSandbox`. [Where and when a policy applies](#) covers Cowork and cloud sessions.

If you're the administrator, [Set up Claude Code for your organization](#) walks through choosing what to enforce, and [Deploy managed settings](#) covers delivery and how to confirm a policy is in force.

Change a setting

You can change a setting from the `/config` menu, by editing a settings file, or for one session from the command line.

Claude Code's system prompt isn't published. To give Claude standing instructions, use [CLAUDE.md files](#) or the `-append-system-prompt` flag.

Use the /config menu

Run `/config` inside Claude Code and open the **Config** tab. It lists a short set of personal options such as theme, editor mode, and verbose output, not every settings key. Select an option to change it; Claude Code saves it for you:

- **Most options:** `~/.claude/settings.json`
- **A few options, such as Show tips:** `.claude/settings.local.json`
- **The global config options:** `~/.claude.json`

To set one option without the menu, pass `key=value`, such as `/config verbose=true`.

❗ `/config` is part of the terminal interface. The [VS Code](#) chat panel and the [desktop app](#) don't open it; change settings there by editing a settings file or through those apps' own settings.

Edit a settings file

Open the settings file for the scope you want in your editor and add or change a key. Settings files are strict JSON: a `//` comment or a trailing comma is a syntax error, and Claude Code reports the file as a **Settings Error** at the next start. For example, to let Claude Code run your lint and test commands without asking and stop it reading `.env` files, add this to `~/.claude/settings.json`:

```
"allow": [
  "Bash(npm run lint)",
  "Bash(npm run test *)"
],
"deny": [
  "Read(./env)",
  "Read(./env*)"
]
}
```

Each entry under `permissions` is a rule that names a tool and what it may do; [Configure permissions](#) explains the syntax. The `$schema` line points to the [published JSON schema](#) for Claude Code settings, which gives you autocomplete and inline validation in VS Code, Cursor, and any other editor that supports JSON schema. The schema can lag behind the newest CLI releases, so a validation warning on a recently documented key doesn't mean your configuration is invalid.

After you save, run `/status` inside Claude Code to confirm the file loaded; [Confirm what loaded](#) says what the `Setting sources` line shows and how a broken file is reported.

For a complete personal file, team file, and organization file, each shown with a comment on every key it sets, see the [example settings files](#).

Change a setting for one session

To try a value without saving it, set it when you start Claude Code. The value applies to that session and your settings files stay as they were. You have three ways to do it:

- `--settings` : pass a key as JSON, inline or as a path to a file. Claude Code applies it above your user, project, and local files and below managed settings. It can set any key your user settings file can set; it can't set `Managed` or `Global config` keys.
- **A flag for that key:** some keys have their own flag, such as `--model` for `model` and `--effort` for `effortLevel` .
- **An environment variable:** export the key's paired variable before you run `claude` , such as `ANTHROPIC_MODEL` for `model` .

Each key's entry on the [settings reference](#) lists its per-session overrides and which one takes precedence, so check the entry for the key you want to change.

Commands you run inside a session mostly save your choice: `/config` writes to your settings files, and `/model` and `/effort` save the value as your default for new sessions. Pressing `s` in the `/model` picker switches the model without saving it, and some `/effort` levels, such as `max` and `ultracode` , apply to the current session only; see [Adjust effort level](#).

When edits take effect

Claude Code watches your settings files and reloads them when they change, so it applies most edits to the running session without a restart, including edits to `permissions`, `hooks`, and credential helpers such as `apiKeyHelper`. The reload covers user, project, local, and managed settings, and Claude Code runs the `ConfigChange` **hook** for each settings-file change it detects, not for managed settings that arrive from MDM or the claude.ai console. Managed settings that arrive through MDM or from the claude.ai console reach a running session on a schedule rather than on save; the **delivery table** gives it per source.

Claude Code reads some keys only once, at session start, so an edit to one of them doesn't reach the running session. Admin-side keys that also wait for a restart, such as `requiredMinimumVersion`, are listed under **where and when a policy applies**. The ones you're most likely to edit mid-session:

- `model` : use `/model` to switch mid-session. Each model has its own prompt cache, so the first request after a switch re-reads the whole conversation uncached; see **Switching models**
- `effortLevel` : use `/effort` to change it mid-session
- `outputStyle` : part of the system prompt, so Claude Code applies the edit after `/clear` or a restart

Confirm what loaded

Run `/status` inside Claude Code to see which settings sources are active. The **Status** tab includes a `Setting sources` line that lists each settings file Claude Code loaded for the current session, such as `User settings` or `Project local settings`. When **managed settings** are in effect, the managed settings entry shows in parentheses how they reached your machine.

The line confirms which files Claude Code read; it doesn't show which file supplied each key. To list entries Claude Code rejected, run `claude doctor`; for a model that project or managed settings set, the startup header names the file that set it. `/status` and `/config` open the same dialog on different tabs, and the **Config** tab isn't a view of your `settings.json` contents.

Fix a broken settings file

If you mistype JSON or set a key to a value Claude Code doesn't accept, Claude Code tells you at the start of an interactive session. What it shows depends on how much of the file is affected:

- **Settings Error**: a user, project, or local file has invalid JSON or a value the schema rejects. At the start of an interactive session Claude Code shows a dialog that lets you fix the file with Claude's help, exit, or continue without the broken settings.
- **Settings Warning**: only individual entries fail, such as a malformed permission rule or an unknown hook event name. Claude Code skips those values and keeps the rest of the file in effect.
- **Managed settings**: Claude Code keeps enforcing the rest of the file. **Invalid entries in managed settings** says what it drops and which keys fall back to a stricter value until you fix them.

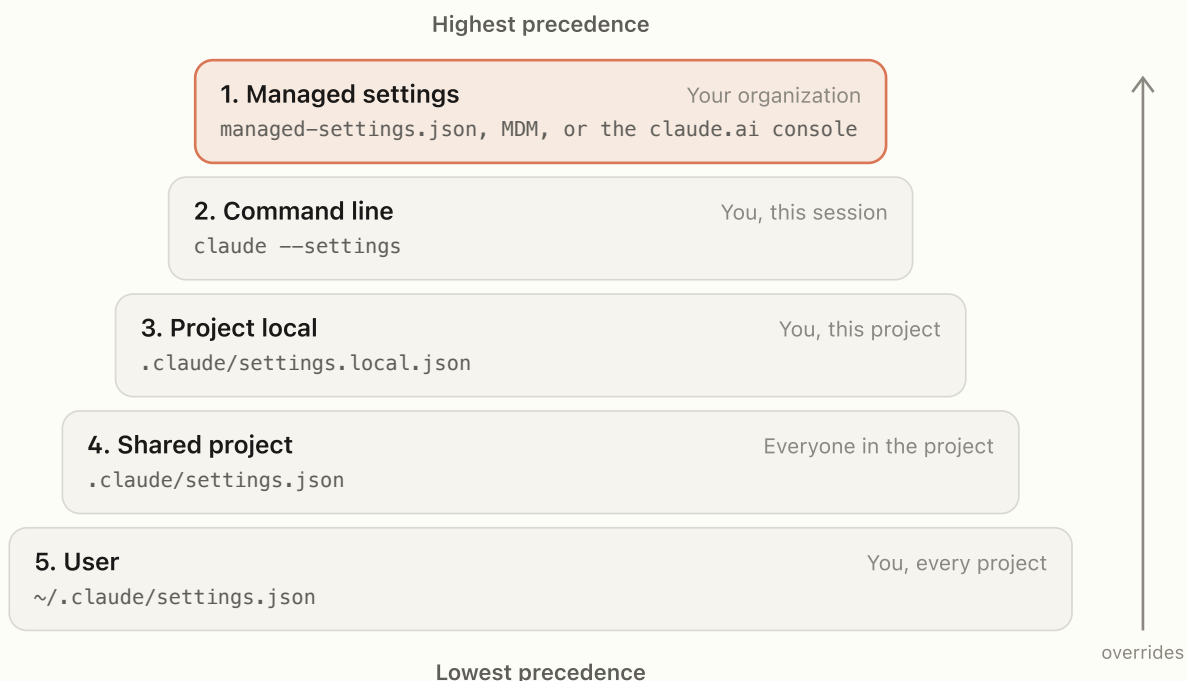
Claude Code saves before it writes the file.

After you continue, run `/status` to see the affected files and `claude doctor` for the details of each error.

A `-p` run shows no dialog: Claude Code skips the broken file or values and continues with the rest, so after a `-p` run that ignores a setting, run `claude doctor` to see what it dropped.

Settings precedence

When the same key appears in more than one place, Claude Code uses the value from the highest level that sets it. The stack below shows the levels, highest on top; a key at a higher level overrides the same key anywhere below it.



In order, highest precedence first:

- Managed settings:** settings your organization deploys, by a `managed-settings.json` file, an MDM policy, or **server-managed settings** from the claude.ai console. Nothing you set overrides them: a key you pass with `--settings` doesn't override the same managed key, and a flag such as `--model` picks only from the models your organization allows. A managed `model` sets the model each session starts with, and you can still switch with `/model`; the lock is `availableModels`, which constrains `/model`, `--model`, and the `model` key in your own files. When your organization delivers more than one managed source, the rules for **precedence within the managed tier** say what Claude Code reads from each.
- Command line arguments:** flags you pass when you start `claude` from a terminal, for one session; see **Change a setting for one session**. Claude Code merges JSON you pass with `--settings <file-or-json>` with your settings files by the same rules as the other levels: it takes a key you set here over the same key in local, project, or user settings, and keeps the lower-level value for a key you omit.

Environment variables aren't a level in this stack. When a behavior has both a shell variable and a settings key, which one applies is decided per pair, not by level: `ANTHROPIC_MODEL` exported in your shell applies over the `model` key from any file, while `ANTHROPIC_DEFAULT_MODEL` applies only when no file sets `model`. The [environment variables reference](#) says which keys have a pair and which one Claude Code reads first. An `env` block inside a settings file is an ordinary key and follows the levels above.

For a few security-sensitive keys, Claude Code honors a stricter value from a lower level over a managed value; [Exceptions to managed settings precedence](#) lists them.

Lists merge instead of overriding

When you set the same list key, such as `permissions.allow`, in more than one file, Claude Code combines the lists instead of picking one, so each file can add entries without removing another file's. Three keys that hold model lists follow their own rules:

- `fallbackModel` is an ordered chain where position carries meaning, so Claude Code takes the whole value from the highest-precedence file that defines it.
- `modelPicker` holds one ordered list of rows plus a replace flag, so Claude Code never merges rows from two sources. It takes the whole value from the highest of managed settings, `--settings`, and user settings that defines it, and ignores the key in project and local settings. Requires Claude Code v2.1.242 or later.
- `availableModels`: when the managed settings Claude Code applies define it, Claude Code applies that list as-is and ignores entries you add in user, project, or local settings, unless an app that embeds Claude Code supplies its own model list; see [Exceptions to managed settings precedence](#). Across managed sources the list never merges either; [how Claude Code combines managed sources](#) says which source's list applies. Across non-managed scopes Claude Code merges the arrays as usual.

Precedence examples

While Claude works, Claude Code shows a one-line tip under the spinner, such as “Use `/config` to change your default permission mode (including Plan Mode)”. Suppose you want those tips off, so you set `spinnerTipsEnabled` to `false` in `~/.claude/settings.json`. Each scenario below is something that can turn them back on, and what you can do about it.

Team settings override personal settings

Your team's `.claude/settings.json` sets it to `true`. Claude Code uses the project value because shared project sits above user, so you see tips in that project and nowhere else.

You can get your value back: add `"spinnerTipsEnabled": false` to `.claude/settings.local.json` in that project. Project local sits above shared project, so your sessions there stop showing tips and your teammates' sessions don't change.

You can't get your value back. Run `/status` to see which managed source applies, and ask your administrator if the policy should change.

The command line overrides your files for one session

You started the session with `claude --settings '{"spinnerTipsEnabled": true}'`. Command line sits above every file except managed, so that session shows tips even though your files say `false`.

You get your value back on the next session; `--settings` lasts one session and doesn't write to any file.

A flag or environment variable sets the same thing

Some keys have a command line flag or an environment variable that overrides the settings value regardless of which file set it: `ANTHROPIC_MODEL` overrides the `model` setting, and `--model` overrides both for a session.

Whether you can get your value back depends on the key: unset the variable or drop the flag, and check the key's entry on the [settings reference](#) and the variable's row on the [environment variables reference](#) for which one Claude Code uses.

Troubleshoot a setting that doesn't apply

When you set a key and Claude Code doesn't behave as if you had, start with `/status` to see which files it loaded, then find your symptom below. [Debug your configuration](#) covers the wider checks, including a clean-configuration test.

A value you set is ignored

Something else is setting the same key, or the file didn't load:

- **A higher level sets it.** Another settings file, a `--settings` flag, or a managed source sets the key above yours; the [stack](#) says which. A flag or environment variable can also override the key on its own, decided key by key; the key's entry on the [settings reference](#) says which one Claude Code uses, and the [env entry](#) covers a managed `env` value versus a shell export.
- **A security key keeps its strict value.** For a few keys Claude Code honors the restrictive value from any file, so a project `true` for `disableClaudeAiConnectors` stays on; see [Exceptions to managed settings precedence](#).
- **The file is broken.** Invalid JSON or a rejected value makes Claude Code skip the file or the entry; see [Fix a broken settings file](#).

A managed change hasn't reached you

Managed sources reach a running session on the schedule in the [delivery table](#), so restart the session first. If `/status` then names a different source than the one your administrator changed, a higher-priority source applies; [How Claude Code combines managed sources](#) gives the order.

Managed , or Global config in the Scope column of the **All settings** index; those keys never apply from the shared file, apart from `autoContinueAtUsageLimit` , which a repository file can still switch off: while the file sets the key and no user, `--settings` , or managed value does, Claude Code reads the setting as off. Global config keys apply only from `~/.claude.json` .

- **The key waits for trust.** `permissions.allow` rules, `permissions.additionalDirectories` , `extraKnownMarketplaces` , and most `env` values apply only after each teammate **trusts the folder**. Until then they still see prompts and don't get plugins from a marketplace the file declares. `deny` and `ask` rules apply right away.

Permission rules combine differently than you expected

- **You chose “Yes, and don't ask again” on a permission prompt but still get prompted for the same tool.** That choice saved an `allow` rule to your local file, and an `allow` rule there doesn't outrank an `ask` rule from a project or managed file; **how permission rules combine** explains the order. In the VS Code extension the approval card lets you pick the destination file, including the project's shared file, which changes the rule for everyone; in the CLI, Claude Code writes only to your local file.
- **Your organization's allow rules still apply alongside yours.** That's expected: Claude Code merges `permissions.allow` across scopes, unless your organization sets `allowManagedPermissionRulesOnly` .

Exceptions to managed settings precedence

For a few security-sensitive keys, Claude Code honors a restrictive value from a scope that otherwise couldn't override managed settings. Find the key in this table to see which value it honors and from where.

Key	Value Claude Code honors	Notes
<code>disableClaudeAiConnectors</code>	<code>true</code> from any scope	Honored even when a managed source sets <code>false</code>
<code>enableArtifact</code>	<code>false</code> from any scope, and <code>disableArtifact: true</code> from any scope	Honored even when a managed source sets <code>true</code> ; nothing turns the Artifact tool back on. Requires Claude Code v2.1.242 or later
<code>isolatePeerMachines</code>	<code>true</code> from any scope	Honored even when a managed source sets <code>false</code>
<code>remoteControlAtStartup</code>	<code>false</code> from <code>.claude/settings.json</code> or <code>.claude/settings.local.json</code>	Honored even when a managed source sets <code>true</code> ; a project or local <code>true</code> is ignored
<code>crossSessionInbound</code>	A stricter value from <code>.claude/settings.json</code> or <code>.claude/settings.local.json</code> , on the <code>accept < hold < refuse</code> ladder	Honored over managed, <code>--settings</code> , and user values; a project or local value that isn't stricter is ignored
<code>useAutoModeDuringPlay</code>	<code>false</code> from any managed source, <code>--settings</code> , <code>~/.claude/settings.json</code> , or <code>.claude/settings.local.json</code>	Honored even when the winning managed source sets <code>true</code> ; a <code>false</code> in <code>.claude/settings.json</code> is ignored

An app that runs Claude Code inside itself and sets `CLAUDE_CODE_PROVIDER_MANAGED_BY_HOST` is also an exception. Claude Code takes that app's model configuration over the `model` , `fallbackModel` , and `modelOverrides` keys from every managed source, and over the model-selection variables in a managed `env` block, such as `ANTHROPIC_MODEL` and the `ANTHROPIC_DEFAULT_*.MODEL` family. Claude Code keeps a managed `availableModels` allowlist in force unless the app supplies its own.

Settings in cloud sessions

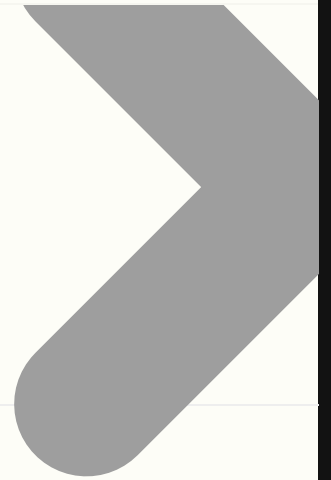
A cloud session, on **Claude Code on the web** or from `claude --cloud` , runs in a **cloud environment** on a fresh clone of your repository, not on your machine. That changes which settings reach it:

- **Shared project settings** (`.claude/settings.json`): read, because the file is part of the clone. Commit a setting there to apply it in cloud sessions.
- **User and project local settings** (`~/.claude/settings.json` and `.claude/settings.local.json`): not read. Both stay on your machine, and the local file isn't in the clone.
- **Managed settings**: only **server-managed settings** reach a cloud session; a `managed-settings.json` file or MDM profile on your device doesn't. A **self-hosted environment** also reads the managed settings file in its runner image. **How Claude Code combines managed sources** says when that file applies.
- `/config` : on the web, opens the Claude Code section of your `claude.ai` settings instead of changing a value. To change a setting for a cloud session, set an **environment variable** on the environment or commit the key to the repository's `.claude/settings.json` .

What carries over from your setup lists the rest: `CLAUDE.md` , skills, MCP servers, plugins, and credentials.

What's next

- **Settings reference**: every key, with where you set it and an example
- **Example settings files**: a personal file, a team file, and an organization's managed file
- **Configure permissions**: allow, ask, and deny rules, and what Claude Code runs without asking
- **Environment variables**: the variables Claude Code reads and the `env` block
- **Debug your configuration**: when a setting doesn't apply
- **Claude directory reference**: every file Claude Code reads, including subagents, MCP servers, plugins, and `CLAUDE.md`



Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Trust center](#)

[Transparency](#)

Learn

[Courses](#)

[MCP connectors](#)

[Customer stories](#)

[Engineering blog](#)

[Events](#)

[Powered by Claude](#)

[Service partners](#)

[Startups program](#)

Help and security

[Availability](#)

[Status](#)

[Support center](#)

Terms and policies

[Privacy choices](#)

[Privacy policy](#)

[Disclosure policy](#)

[Usage policy](#)

[Commercial terms](#)

[Consumer terms](#)